

Pretraining, transfer và mô hình nền tảng (Pretraining, Transfer and Foundation Models)

PHỤ THUỘC B/06-chat-luong-du-lieu và B/07-chia-du-lieu-va-danh-gia (chi phí nhân và cách đánh giá không tự lừa mình); B/09-thiet-ke-ham-mat-mat (*contrastive loss*, cross-entropy); C/13-kien-truc-backbone (ViT và *inductive bias* — chương này giải thích vì sao ViT chỉ toả sáng khi có tiền huấn luyện quy mô lớn).

TÓM TẮT

Chương này trả lời một câu hỏi: nhân thì đắt, dữ liệu không nhân thì vô hạn — vậy tín hiệu giám sát miễn phí lấy từ đâu, và mua bao nhiêu thì đủ. Mục 18.1 đi qua bốn nguồn giám sát rẻ dần và nói mỗi nguồn hỏng ở đâu; mục 18.2 đưa ra mặt định lượng: scaling laws, bài học Chinchilla, nhiệm vụ kiểm tra, và LoRA. Đọc xong bạn chọn được *backbone*, chọn được chế độ thích nghi, và ước lượng được hoá đơn compute — thay vì hỏi người khác hoặc thử mù. Nếu chỉ nhớ một điều: mô hình nền tảng đã biến bạn từ người huấn luyện thành người tích hợp, nên câu hỏi trung tâm đổi từ “mô hình này có đúng không” sang “tôi có biết nó giả định gì và hỏng ở đâu không”.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình (model)** — một hàm có tham số, ánh xạ đầu vào (ảnh) sang đầu ra (nhân, hộp, vector đặc trưng). “Huấn luyện” nghĩa là tìm bộ tham số làm hàm mất mát nhỏ nhất trên dữ liệu.
- **tiền huấn luyện (pretraining)** — huấn luyện một mô hình trên một tập dữ liệu lớn, tổng quát, *trước* khi biết bài toán cuối là gì. Mục đích là để mô hình học sẵn những đặc trưng chung của ảnh, thứ mà dữ liệu ít ỏi của bài toán cuối không đủ để học lại.
- **học chuyển giao (transfer learning)** — lấy mô hình đã tiền huấn luyện rồi dùng lại cho bài toán khác, hoặc bằng cách đóng băng nó, hoặc bằng cách huấn luyện tiếp trên dữ liệu của mình.
- **backbone** — phần thân của mạng chịu trách nhiệm biến ảnh thành vector đặc trưng; phần “đầu” (head) gắn sau nó mới sinh ra đầu ra cụ thể của từng bài toán. Tách hai phần ra là điều kiện để tái sử dụng.
- **học tự giám sát (self-supervised learning)** — huấn luyện mà không dùng nhãn người: đáp án của bài toán phụ được suy ra từ chính dữ liệu, chẳng hạn “hai lát cắt của cùng bức ảnh phải cho vector giống nhau”.
- **mô hình nền tảng (foundation model)** — một mô hình lớn được tiền huấn luyện trên dữ liệu quy mô rất lớn, đủ tổng quát để dùng lại cho nhiều bài toán khác nhau mà không huấn luyện lại từ đầu.
- **VLM (vision-language model)** — mô hình nhận cả ảnh lẫn văn bản làm đầu vào và sinh ra văn bản, thường bằng cách nối một bộ mã hoá ảnh vào một mô hình ngôn ngữ lớn.
- **ngân sách compute** — tổng số phép tính (FLOPs) bạn mua được cho một lần huấn luyện. Vì nó cố định, mọi quyết định về kích thước mô hình và lượng dữ liệu đều là quyết định phân bổ trong cùng một ngân sách.

CÂU HỎI MỞ ĐẦU

- Q1. Với 500 ảnh nội soi, *linear probe* trên một *backbone* khổng lồ hay tinh chỉnh toàn bộ một ResNet nhỏ sẽ thắng? Và vì sao câu trả lời đảo chiều ở 50 000 ảnh?
- Q2. Nếu chỉ cần dạy mô hình rằng hai lát cắt của cùng một bức ảnh phải cho vector giống nhau, vì sao nó không học ngay nghiệm rẻ nhất là xuất một hằng số cho mọi ảnh?
- Q3. Vì sao một dòng chú thích cầu thả trên internet lại là tín hiệu giám sát ngữ nghĩa mạnh hơn một nhãn ImageNet do người gán cẩn thận?
- Q4. Gopher lớn gấp bốn lần Chinchilla và tốn cùng số FLOPs để huấn luyện, nhưng thua — cả ngành đã tối ưu nhằm đạt lượng nào suốt mấy năm?
- Q5. Vì sao một mô hình chưa bao giờ nhìn thấy ảnh vệ tinh lại cho đặc trưng ảnh vệ tinh tốt hơn một mạng huấn luyện riêng trên ảnh vệ tinh?

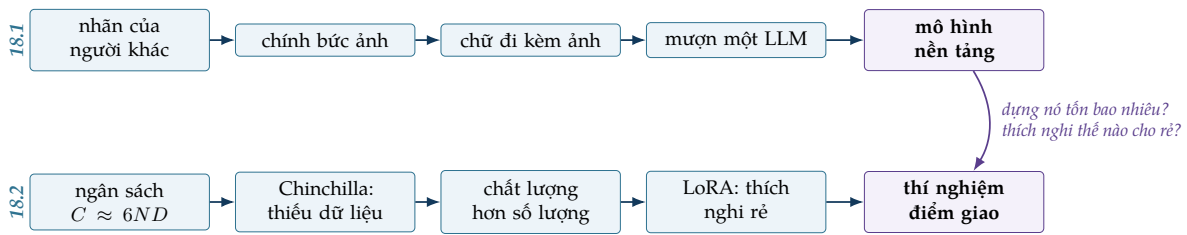
Câu hỏi dẫn dắt của cả chương chỉ có một dòng: *nhân thì đắt, dữ liệu không nhân thì vô hạn* — *tín hiệu giám sát miễn phí có thể lấy từ đâu?* Mọi kỹ thuật trong chương, từ học chuyển giao những năm 2014 tới VLM ngày nay, là một câu trả lời cho đúng câu hỏi đó, và các câu trả lời xếp thành một chuỗi ngày càng táo bạo: dùng nhãn của người khác, rồi dùng chính bức ảnh, rồi dùng dòng chữ đi kèm bức ảnh, rồi mượn luôn khả năng suy luận của một mô hình ngôn ngữ.

Chuỗi ấy là một mạch *định tính*: nó nói cho bạn biết có những nguồn giám sát nào và mỗi nguồn giả định gì. Nhưng nó không trả lời được câu hỏi mà bạn thực sự phải quyết định khi ngồi trước một cụm GPU có hạn: *tiêu ngân sách vào đâu, và bao nhiêu thì đủ?* Đó là một mạch khác, mang tính *định lượng* và kinh tế, với từ vựng riêng — *scaling laws*, tỉ lệ dữ liệu trên tham số, chi phí thích nghi. Hai mạch dùng chung đối tượng nhưng khác hẳn nhau về loại lập luận, và trộn chúng vào một mạch duy nhất là cách nhanh nhất để cả hai đều mờ đi. Vì vậy chương chia làm hai mục:

- **18.1 — Nguồn giám sát ngày càng rẻ.** Chuỗi bốn nguồn, dừng ở từng nguồn để hỏi nó chống lại nghiệm tầm thường bằng cách nào, nó ép vào biểu diễn những bất biến gì, và nó hỏng ở đâu.
- **18.2 — Kinh tế học của pretraining.** Lấy kết quả của mục trước làm cho sẵn rồi hỏi tiếp: với chừng này compute thì mô hình nên lớn cỡ nào, dữ liệu nên nhiều cỡ nào, và khi nào tinh chỉnh đáng giá hơn đóng băng.

Thứ tự đọc là bắt buộc theo chiều này: mục 18.2 nhắc tới *linear probe*, *fine-tune* và mô hình nền tảng như những khái niệm đã biết. Có một điểm nối giữa hai mục đáng nói trước: cả hai kết thúc ở cùng một kết luận về phương pháp luận. Mục 18.1 kết luận rằng bạn không thể biết một biểu diễn tự giám sát đã học gì nếu chỉ nhìn *benchmark* chung; mục 18.2 kết luận rằng bạn không thể biết chế độ thích nghi nào đúng nếu chỉ nghe lời khuyên chung. Cả hai đẩy bạn về cùng một hành động: đo trên chính dữ liệu của mình, với một tập đánh giá mà mô hình nền tảng chắc chắn chưa từng thấy.

Hình 1.1 vẽ chỗ nối giữa hai mục: mục 18.1 sản xuất ra một mô hình nền tảng, còn mục 18.2 hỏi cái mô hình ấy tốn bao nhiêu và thích nghi thế nào cho rẻ.



Hàng trên: giám sát rẻ dần, tín hiệu gián tiếp dần. Hàng dưới: cùng một ngân sách nhìn từ phía chi phí.

Hình 1.1. Bản đồ chương 18. Hai mục không nối tiếp nhau về thời gian mà nối nhau ở *đối tượng*: mục 18.1 kết thúc ở mô hình nền tảng, và mục 18.2 bắt đầu bằng câu hỏi mô hình đó tốn bao nhiêu. Mọi con số chung ở hàng dưới đều kết thúc ở cùng một chỗ — một thí nghiệm bạn phải tự chạy trên dữ liệu của mình.

Bản đồ chương 18 — Pretraining, transfer và mô hình nền tảng

Chương này trả lời một câu hỏi duy nhất — *tín hiệu giám sát miễn phí lấy từ đâu, và mua bao nhiêu thì đủ* — nhưng câu hỏi đó có hai nửa dùng hai loại lập luận khác nhau, nên chương chia làm hai mục và thứ tự đọc là bắt buộc.

18.1 — Nguồn giám sát ngày càng rẻ [T3] (cốt lõi). Chuỗi bốn nguồn giám sát xếp theo chi phí giảm dần: nhãn của người khác (transfer learning và bảng bốn ô quyết định), chính bức ảnh (ba triết lý tự giám sát và ba cách chống sập khác nhau), văn bản đi kèm ảnh (CLIP và ngữ nghĩa mở), và cuối cùng là ghép thị giác vào một LLM (VLM). Đây là mục phải nắm chắc: nó là tiền đề trực tiếp của open-vocabulary detection ở chương 14 và của mọi quyết định chọn backbone về sau. Điểm cần mang theo là cột “hồng khi nào” của bảng ba họ tự giám sát, và ý thức rằng mỗi bất biến bạn ép vào augmentation là một loại thông tin bạn cố tình vứt đi.

18.2 — Kinh tế học của pretraining [T2][T3] (cốt lõi phân nguyên lý, tham khảo phân con số). Mặt định lượng: scaling laws cho phép ngoại suy thay vì thử mù; bài học Chinchilla cho thấy cả ngành đã thổi tham số mà quên dữ liệu; chất lượng corpus thắng số lượng; contamination phá hủy hiệu lực của mọi benchmark công khai; PEFT/LoRA đổi hoàn toàn kinh tế học của việc thích nghi; và quyết định cuối cùng luôn là thí nghiệm điểm giao trên dữ liệu của chính bạn. Nguyên lý ở đây thuộc tầng bền ([T2]); các hằng số cụ thể — tỉ lệ 20 token trên tham số, hạng LoRA hợp lý — thì không, và nên tra lại thay vì nhớ.

Vì sao chia như vậy. Mục 18.1 trả lời “có những gì và nó giả định gì”; mục 18.2 trả lời “tiêu bao nhiêu vào đâu”. Nếu gộp lại, phân định lượng sẽ nuốt mất phân định tính — người đọc nhớ con số mà quên cơ chế. Mục 18.2 dùng lại từ vựng của 18.1 (linear probe, fine-tune, mô hình nền tảng) như đã biết, nên đọc ngược sẽ vấp.

Phụ thuộc lên trên. B/06-chat-luong-du-lieu (vì sao nhãn đắt và bẩn), B/07-chia-du-lieu-va-danh-gia (vì sao contamination là vấn đề chết người), B/09-thiet-ke-ham-mat-mat (contrastive loss), C/13-kien-truc-backbone/03-vit-va-inductive-bias (vì sao ViT cần pretraining lớn).

Toả xuống dưới. C/14-bai-toan-cv-loi kế thừa trực tiếp không gian nhúng CLIP; D/20-toi-uu-va-nen-mo-hinh và D/21-inference-va-trien-khai nhận lấy mô hình nền tảng như một ràng buộc phải nín; E/24-thi-nghiem-va-ablation dạy cách chạy thí nghiệm điểm giao mà không tự lừa mình.

1.1 Nguồn giám sát ngày càng rẻ (Increasingly Cheap Supervision)

PHỤ THUỘC B/06-chat-luong-du-lieu (nhân đất và bắn tới mức nào); B/09-thiet-ke-ham-mat-mat (*contrastive loss*, cross-entropy); C/13-kien-truc-backbone/03-vit-va-inductive-bias (vi sao ViT cần nhiều dữ liệu hơn CNN); C/14-bai-toan-cv-loi (*open-vocabulary detection* là hệ quả trực tiếp của mục này).

TÓM TẮT

Mục này xếp bốn nguồn giám sát theo chi phí giảm dần: nhãn của người khác, chính bức ảnh, dòng chữ đi kèm ảnh, và khả năng suy luận mượn từ một mô hình ngôn ngữ. Với mỗi nguồn, mục nói rõ nó chống nghiệm tầm thường bằng cơ chế nào và hỏng ở đâu. Đọc xong, trước một bài toán có vài trăm ảnh nhãn, bạn quyết được ngay ba việc: đóng băng hay tinh chỉnh, chọn backbone tự giám sát nào, và công thức augmentation nào sẽ phá hỏng đúng tín hiệu bạn cần. Đoán mò ba việc đó thì mỗi cái tốn vài ngày GPU mới biết là sai. Nếu chỉ nhớ một điều: mỗi bất biến bạn ép vào augmentation chính là một loại thông tin bạn cố tình vứt đi, nên hãy liệt kê các bất biến đó trước khi sao chép công thức của bất kỳ paper nào.

KIẾN THỨC CẦN CÓ TRƯỚC

- **nhãn (label) và tín hiệu giám sát** — nhãn là đáp án đúng gắn với một mẫu dữ liệu; tín hiệu giám sát là bất cứ thứ gì đóng vai trò đáp án để tính hàm mất mát. Mọi ý tưởng trong mục này là cách tạo tín hiệu giám sát mà không phải trả tiền cho nhãn.
- **đặc trưng (feature) và embedding** — đặc trưng là vector số mà mạng sinh ra để mô tả một bức ảnh; *embedding* là cách gọi khác khi ta quan tâm tới hình học của không gian đó — ảnh giống nhau thì vector gần nhau.
- **đóng băng (frozen) và linear probe** — đóng băng là giữ nguyên trọng số backbone, không cho gradient cập nhật. *Linear probe* là huấn luyện đúng một tầng tuyến tính trên đặc trưng đóng băng — phép thử rẻ nhất để xem đặc trưng có chứa thông tin cần thiết hay không.
- **fine-tune (tinh chỉnh)** — huấn luyện tiếp một mô hình đã có trọng số, thường với *learning rate* nhỏ hơn nhiều lần để không xoá mất thứ nó đã học.
- **augmentation** — biến đổi ngẫu nhiên áp lên ảnh lúc huấn luyện (cắt, lật, đổi màu, làm mờ) nhằm dạy mô hình rằng những biến đổi đó không làm đổi đáp án. Chính vì thế nó cũng *xoá* thông tin: bất biến nào bạn dạy thì thông tin đó mất.
- **contrastive (tương phản)** — kiểu học kéo các cặp “giống nhau” lại gần và đẩy các cặp “khác nhau” ra xa trong không gian đặc trưng. Cặp giống nhau gọi là *positive*, cặp khác nhau gọi là *negative*.
- **masked modeling** — che một phần đầu vào rồi bắt mô hình khôi phục phần bị che. Vì đáp án chính là dữ liệu gốc, không cần nhãn nào cả.
- **zero-shot** — dùng mô hình cho một lớp hoặc một tác vụ mà nó chưa từng được huấn luyện trên đó, chỉ bằng cách mô tả lớp ấy bằng ngôn ngữ.

A. Đặt vấn đề

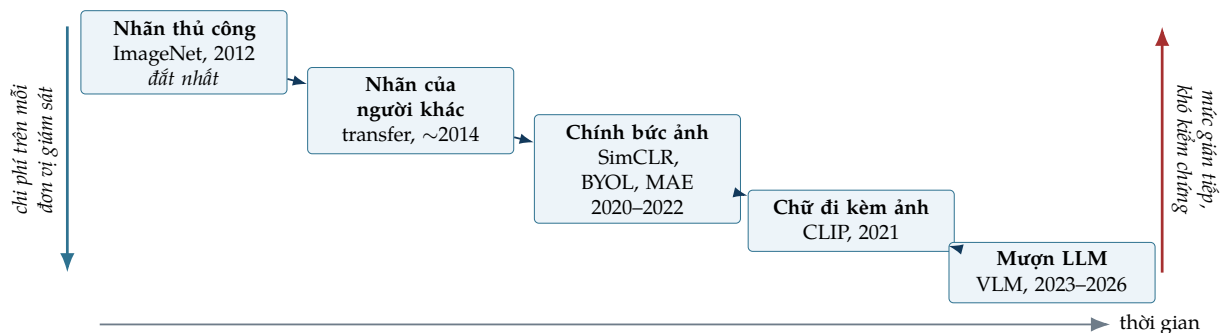
Nhãn đất, dữ liệu không nhãn thì gần như vô hạn. Một bộ ảnh nội soi có nhãn tổn thương cần bác sĩ ngồi vẽ *mask*; một bộ ảnh vệ tinh có nhãn loại đất cần chuyên gia đối chiếu thực địa. Trong khi đó, ổ cứng của cùng bệnh viện hay cùng cơ quan trắc địa chứa hàng triệu ảnh chưa ai chạm vào. Toàn bộ mục này là chuỗi câu trả lời ngày càng táo bạo cho một câu hỏi duy nhất: *tín hiệu giám sát miễn phí có thể lấy từ đâu?*

Câu hỏi nghe như một mẹo tiết kiệm chi phí, nhưng nó thực ra là câu hỏi về bản chất của việc học. Nhãn do người viết ra không phải thứ duy nhất mang thông tin về thế giới. Thông tin còn nằm ở ba chỗ khác. Hai bức ảnh chụp cùng một vật phải cho đặc trưng giống nhau. Phần bị che của một bức ảnh đoán được từ phần còn lại. Và dòng chú thích dưới một bức ảnh trên internet nói về đúng bức ảnh đó. Mỗi lần ngành khai thác được một nguồn như vậy, chi phí trên mỗi đơn

vị giám sát giảm một bậc. Cái giá cũng tăng một bậc: tín hiệu gián tiếp hơn, nhiều hơn, và khó truy ra mô hình đã học gì hơn.

Học xong mục này thì làm được gì mà trước đó không làm được? Bạn nhìn một bài toán có 300 ảnh nhân và quyết định ngay: đóng băng hay tinh chỉnh, chọn *backbone* tự giám sát nào, và công thức *augmentation* nào sẽ phá hỏng chính tín hiệu bạn cần. Đoán mò ba quyết định đó thì mỗi cái tốn vài ngày GPU mới lộ ra là sai.

Hình 1.2 đặt cả chuỗi lên một trục thời gian trước khi ta đi vào từng bước. Điều đáng nhìn không phải các mốc năm mà là *hai mũi tên ngược chiều nhau* ở hai bên: chi phí đi xuống trong khi mức gián tiếp đi lên. Toàn bộ chương là câu chuyện về việc đánh đổi giữa hai trục này.



Hình 1.2. Bốn nguồn giám sát trên một trục thời gian. Mỗi bước đi xuống một bậc chi phí và đi lên một bậc gián tiếp; không bước nào miễn phí thật sự, chúng chỉ dời hoá đơn từ “thuê người gắn nhãn” sang “không biết mô hình đã học gì”.

TRỰC GIÁC

Nghĩ về nó như bài toán trắc địa. Để dựng một bản đồ, bạn không cần toạ độ tuyệt đối của từng cột mốc — một việc đắt như thuê máy đo GNSS cho mỗi điểm. Bạn chỉ cần đủ nhiều *ràng buộc tương đối* giữa các cột mốc (khoảng cách, góc), rồi giải hệ; hình dạng mạng lưới hiện ra chính xác, chỉ còn thiếu một phép biến đổi toàn cục — *gauge* — mà một vài điểm chuẩn rẻ tiền là ghim lại được.

Học tự giám sát làm đúng như vậy trong không gian đặc trưng. Mỗi cặp *augmentation* của cùng một ảnh là một ràng buộc “hai điểm này phải gần nhau”; mỗi cặp ảnh khác nhau là ràng buộc “hai điểm này phải xa nhau”. Giải cả hệ ra được một không gian đặc trưng có hình dạng đúng nhưng chưa gắn nhãn — tức là xác định tới một *gauge*. Vài trăm ảnh có nhãn sau đó đóng vai trò các điểm chuẩn: *linear probe* chính là bước ghim *gauge*, và nó rẻ vì nó chỉ phải xác định một phép biến đổi chứ không phải toàn bộ hình dạng.

Mạch chính — bốn nguồn giám sát, theo thứ tự rẻ dần

- 1 Vấn đề** gắn nhãn cho bài toán của mình quá đắt, nhưng người khác đã gắn nhãn cho bài toán của họ.

Giải pháp **học chuyển giao** (*transfer learning*): lấy trọng số đã học trên ImageNet [1] làm điểm khởi đầu thay vì khởi tạo ngẫu nhiên.

Giả định các tầng đầu của mạng học những thứ chung cho mọi ảnh tự nhiên — cạnh, kết cấu, gradient màu — nên chúng không phụ thuộc vào việc nhãn đầu ra là gì.

Giá phải trả bạn thừa hưởng cả những thiên lệch của nguồn, kể cả những thiên lệch bạn không muốn.
- 2 Vấn đề** ngay cả nhãn của người khác cũng có giới hạn, và miền của bạn có thể chẳng ai gắn nhãn.

Giải pháp	học tự giám sát (<i>self-supervised learning</i>): tạo bài toán phụ mà đáp án suy ra được từ chính bức ảnh.
Giả định	cấu trúc thống kê của ảnh đủ giàu để ép mô hình học biểu diễn có nghĩa khi ta buộc nó giải bài toán phụ đó.
Vấn đề mới	bài toán phụ có nghiệm tầm thường — mô hình xuất ra một hằng số là thoả mọi ràng buộc “giống nhau”. Chống lại sụp biểu diễn (<i>collapse</i> : mọi ảnh cho cùng một vector) này là toàn bộ nội dung kỹ thuật của mảng này.
3 Vấn đề	ảnh tự nó không nói cho ta biết ngữ nghĩa; nó chỉ nói về tương quan pixel.
Giải pháp	CLIP và tiền huấn luyện thị giác-ngôn ngữ (<i>vision-language</i>) [2]: dùng dòng chú thích đi kèm ảnh trên internet làm giám sát.
Hệ quả	ngôn ngữ trở thành nguồn giám sát ngữ nghĩa rẻ nhất từng có, và nó mang theo một tập nhãn <i>mở</i> thay vì 1000 lớp cố định.
Giá phải trả	dữ liệu web bẩn, thiên lệch, và không kiểm toán được.
4 Vấn đề	mô hình vẫn chỉ nhận dạng, chưa suy luận.
Giải pháp	mô hình thị giác-ngôn ngữ (<i>vision-language model, VLM</i>) — nối một <i>vision encoder</i> vào một LLM đã biết suy luận qua một tầng chiếu hoặc <i>cross-attention</i> [3], [4].
Giả định	biểu diễn thị giác có thể được “phiên dịch” thành thứ mà không gian <i>token</i> của LLM hiểu được, và phần suy luận không cần học lại.
Giá phải trả	đắt, khó kiểm soát, và ảo giác (<i>hallucination</i>) của phần ngôn ngữ lan sang phần thị giác.
Hệ quả	tích lũy của cả bốn bước: mô hình nền tảng (<i>foundation model</i>) trở thành một <i>thành phần dựng sẵn</i> , giống thư viện BLAS chứ không giống một mô hình. Nhiều khi dùng đặc trưng đã tiền huấn luyện tốt hơn hẳn tự train, <i>kể cả trong lĩnh vực hẹp</i> như ảnh y tế hay ảnh vệ tinh. Điều này trái trực giác, nhưng đã lặp lại đủ nhiều lần để tin [5].

B. Nguồn 1 — nhãn của người khác (Transfer Learning)

Học chuyển giao là kỹ thuật cũ nhất và vẫn là kỹ thuật đáng thử đầu tiên. Điều đáng nói không phải “nó hoạt động”, mà là *quyết định thực tế gói gọn trong một bảng bốn ô*. Học thuộc bảng này tiết kiệm rất nhiều thí nghiệm vô ích.

Bảng 1.1. Bốn ô quyết định của học chuyển giao. Trục dọc là khoảng cách miền, trục ngang là lượng dữ liệu có nhãn của bạn.

	Dữ liệu ít	Dữ liệu nhiều
Miền giống nguồn	<i>Linear probe</i> trên đặc trưng đóng băng	<i>Fine-tune</i> toàn bộ
Miền khác nguồn	<i>Fine-tune</i> vài tầng cuối, <i>augmentation</i> mạnh	<i>Fine-tune</i> toàn bộ; cân nhắc train từ đầu

Nói gọn thì logic bên dưới Bảng 1.1 chỉ có một câu: *số tham số bạn cho phép thay đổi phải tương xứng với lượng thông tin mới mà dữ liệu của bạn mang tới*. Đây chính là trục đánh đổi giả định–dữ liệu đã gặp ở B/04-nen-tang-hoc-tu-du-lieu, lần này xuất hiện dưới dạng “mở bao nhiêu tầng”. Ba hệ quả cụ thể:

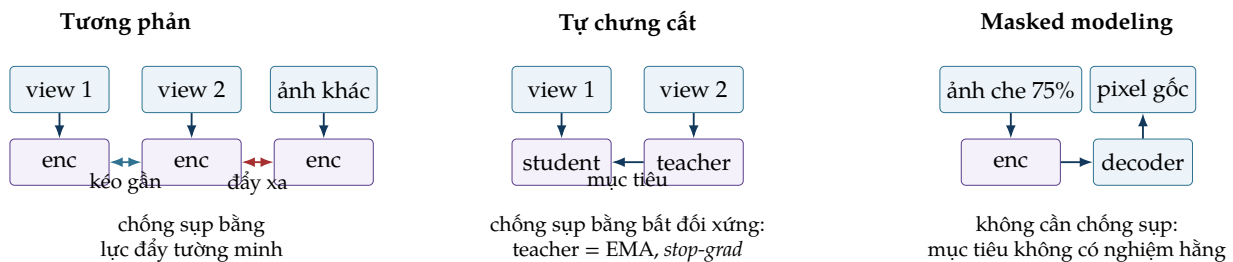
↪ **cùng nguyên lý với** B/04 — đây chính là trục đánh đổi giả định–dữ liệu, lần này xuất hiện dưới dạng “mở bao nhiêu tầng thì vừa với lượng nhãn đang có.

- **Dữ liệu ít** $\Rightarrow$ **mở ít**. Mở toàn bộ mạng là mời gọi *overfitting* và **quên thảm khốc** (*catastrophic forgetting*) — mạng quên đặc trưng tổng quát để chạy theo vài trăm mẫu.
- **Miền xa** $\Rightarrow$ **thay tầng cuối**. Các tầng cuối chứa thông tin chuyên biệt của nguồn nên phải thay; các tầng đầu vẫn dùng được vì cạnh vẫn là cạnh.
- **Cả hai điều kiện thuận** $\Rightarrow$ **tiền huấn luyện chỉ còn là khởi tạo tốt**. Khi bạn có nhiều dữ liệu và miền xa, lợi ích của tiền huấn luyện thu hẹp dần và đôi khi biến mất.

Một chi tiết dễ bị bỏ qua: chất lượng mô hình nguồn trên ImageNet có tương quan với chất lượng chuyển giao, nhưng tương quan đó *yếu đi rõ rệt* khi miền đích xa [6]. Nói cách khác, đừng chọn *backbone* chỉ vì nó đứng cao trên bảng ImageNet; với ảnh nội soi hay ảnh radar, thứ hạng đó gần như không mang thông tin.

C. Nguồn 2 – chính bức ảnh (Self-Supervised Learning)

Học tự giám sát tách thành ba triết lý khác nhau về cùng một câu hỏi: làm sao tạo tín hiệu từ hư không mà không rơi vào nghiệm tầm thường. Hình 1.3 cho thấy ba *cấu trúc đồ thị tính toán* khác nhau, và chính sự khác nhau về cấu trúc — chứ không phải về **hàm mất mát** (*loss function*) — mới là điều đáng nhớ.



Hình 1.3. Ba triết lý tự giám sát khác nhau ở *cấu trúc đồ thị*, không ở hàm mất mát. Tương phản chống sụp bằng lực đẩy từ *negative*; tự chưng cất chống sụp bằng bất đối xứng kiến trúc; *masked modeling* không cần chống sụp vì khôi phục pixel không có nghiệm hằng số.

Bảng 1.2. Ba họ tự giám sát. Cột cuối là cột đáng tiền nhất.

Triết lý	Cơ chế chống sụp	Giả định ngầm	Hỏng khi nào
Tương phản (<i>contrastive</i>) (SimCLR, MoCo)	Lực đẩy tường minh từ rất nhiều <i>negative</i>	Hai ảnh khác nhau trong <i>batch</i> thì thực sự khác lớp	Dữ liệu ít lớp hoặc lặp nhiều: <i>negative</i> hoá ra là <i>positive</i> ; <i>batch</i> nhỏ làm tín hiệu yếu
Tự chưng cất (<i>self-distillation</i>) (BYOL, DINO)	Bất đối xứng kiến trúc: teacher <i>EMA</i> (trung bình trượt, cập nhật chậm theo student), <i>stop-gradient</i> , <i>centering</i>	Điểm cố định của động lực học huấn luyện không phải hằng số	Cơ chế mong manh: đổi tốc độ <i>EMA</i> hay bỏ <i>predictor</i> là sụp; rất nhạy siêu tham số
Masked modeling (MAE)	Không cần: khôi phục pixel không có nghiệm tầm thường	Ngữ nghĩa nằm trong khả năng nội suy phần bị che	Đặc trưng thô, <i>linear probe</i> kém; phải <i>fine-tune</i> mới mạnh — tức đắt hơn ở phía sau

Bảng 1.2 nói rằng ba họ không cạnh tranh trên cùng một trục. Tương phản [7], [8] và tự chưng cất [9], [10] cho đặc trưng dùng được ngay với *linear probe*, hợp khi bạn định đóng băng *backbone*. Masked modeling [11] cho đặc trưng cần tinh chỉnh, hợp khi bạn có đủ dữ liệu đích và muốn tận dụng khả năng mở rộng của ViT.

Ví dụ nhỏ nhất: nhiệt độ quyết định gradient nằm ở đâu

Lấy một *batch* bốn ảnh. Đặc trưng đã chuẩn hoá, **độ tương đồng cosine** (*cosine similarity*) của cặp dương là 0,9 và của ba cặp âm đều là 0,1. Hàm InfoNCE [12] cho một mẫu là

$$L = -\log \frac{e^{s^+/\tau}}{e^{s^+/\tau} + \sum_j e^{s_j^-/\tau}}.$$

Đọc bằng lời: đây là cross-entropy của một bài toán phân loại mà “lớp đúng” là bạn đồng hành của chính mình, còn τ quyết định phân phối đó nhọn tới mức nào. Thay số cho ba giá trị **nhiệt độ** (*temperature*):

- $\tau = 1$: $L = -\log(2,460/5,775) = 0,853$
- $\tau = 0,5$: $L = -\log(6,050/9,714) = 0,474$
- $\tau = 0,1$: $L = -\log(8103,1/8111,2) = 0,0010$

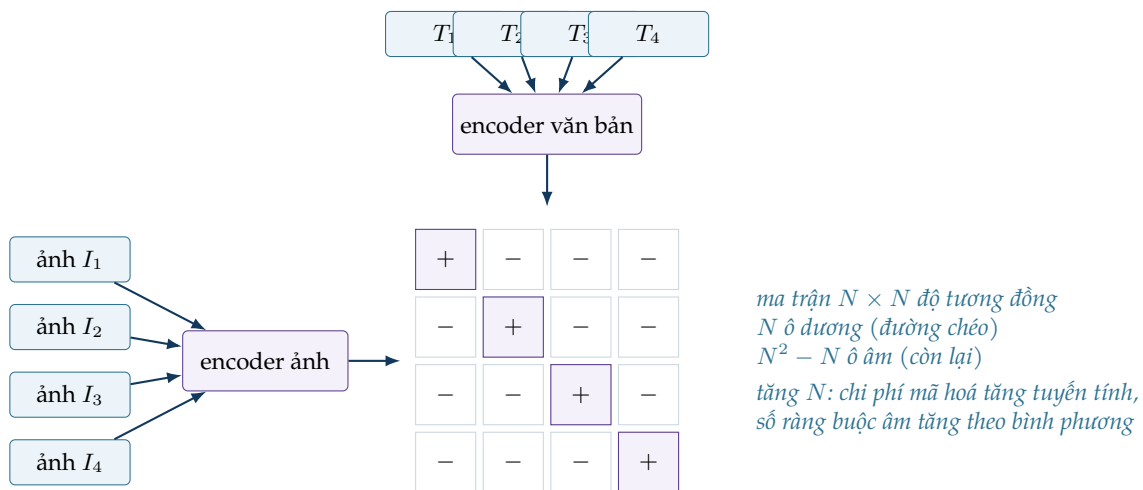
Ba con số nói rõ một điều mà công thức tổng quát giấu đi. Ở nhiệt độ thấp, một cặp đã tách được ở mức tầm thường (0,9 so với 0,1) đóng góp gần như *không* gradient; toàn bộ gradient dồn vào những *negative* có độ tương đồng gần với *positive*. Nhiệt độ vì thế không phải siêu tham số trang trí — nó là núm vặn quyết định hàm mất mát chú ý tới *negative* khó tới mức nào. Vặn quá thấp thì mô hình chỉ học từ vài cặp khó nhất và trở nên nhạy với nhiễu ngẫu nhiên; vặn quá cao thì mọi cặp đóng góp như nhau và biểu diễn không tách được các lớp gần nhau. Đây đúng là cơ chế đã gặp ở B/09-thiet-ke-ham-mat-mat dưới tên *hard negative mining*, chỉ khác là ở đây nó được điều khiển bằng một số thực thay vì bằng một bộ lọc.

↔ cùng nguyên lý với B/09 — nhiệt độ dồn gradient về các cặp khó chính là hard negative mining, chỉ khác là ở đây nó được điều khiển bằng một số thực thay vì bằng một bộ lọc.

D. Nguồn 3 — văn bản đi kèm ảnh (Vision-Language Pretraining)

CLIP giữ nguyên ý tưởng tương phản nhưng đổi định nghĩa cặp dương: thay vì hai *augmentation* của cùng một ảnh, cặp dương là (ảnh, dòng chú thích của chính nó), và cặp âm là mọi tổ hợp chéo còn lại trong *batch*. Với *batch* kích thước N , một lượt tính cho ma trận $N \times N$ độ tương đồng và $2N$ bài toán phân loại N lớp — đó là lý do CLIP thêm *batch* lớn tới mức bất thường.

Hình 1.4 cho thấy vì sao một *batch* lớn lại quan trọng đến thế: mỗi lượt chỉ có N ô dương trên đường chéo nhưng $N^2 - N$ ô âm, nên tăng N làm số ràng buộc âm tăng theo bình phương trong khi chi phí mã hoá chỉ tăng tuyến tính.



Hình 1.4. CLIP đối định nghĩa cặp dương: thay vì hai *augmentation* của cùng một ảnh, cặp dương là (ảnh, chú thích của chính nó). Một lượt cho N bài toán phân loại theo hàng và N bài theo cột trên cùng một ma trận — đó là toàn bộ lý do CLIP thêm *batch* lớn bất thường.

Nhận thức then chốt không nằm ở kiến trúc mà ở tính chất của tín hiệu:

- **Nhãn ImageNet** là một hàm từ ảnh vào một tập 1000 phần tử *cố định*.
- **Chú thích văn bản** là một hàm vào một không gian ngữ nghĩa *mở*.

Hệ quả trực tiếp là phân loại *zero-shot*: muốn thêm một lớp mới, bạn viết thêm một câu chứ không gán thêm nhãn. Hệ quả xa hơn là toàn bộ dòng **phát hiện từ vựng mở** (*open-vocabulary detection*) và phân vùng ở chương 14. Chúng chỉ ghép bộ đề xuất vùng với không gian *embedding* văn bản này, nên mọi điểm yếu của CLIP di truyền nguyên vẹn sang đó.

↪ **hệ quả của C/14** — phát hiện từ vựng mở là bộ đề xuất vùng cắm vào chính không gian embedding này, nên nó thừa hưởng nguyên vẹn điểm yếu của CLIP về quan hệ không gian và đếm.

Cái giá thì cụ thể và đã được đo nhiều lần. Mô hình học từ chú thích web hành xử gần với một túi từ hơn là một bộ phân tích cú pháp: nó phân biệt tốt “có con chó” với “có con mèo”, nhưng yếu ở quan hệ không gian (“con mèo bên trái con chó”), ở đếm số lượng, và ở phủ định. Kết quả *zero-shot* còn nhạy với cách viết *prompt* tới mức khó chịu. Chênh lệch giữa “a photo of a {}” và “{}” có thể tới vài điểm phần trăm. Hãy đọc con số đó như dấu hiệu mô hình chưa ổn định, không phải như một mẹo tối ưu.

E. Nguồn 4 — ghép thị giác vào một LLM (Multimodal LLM)

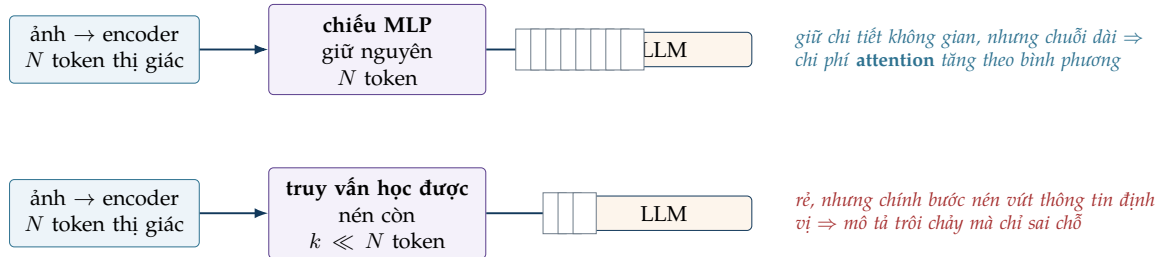
Bước tiếp theo hỏi: nếu đã có một mô hình ngôn ngữ biết suy luận, tại sao phải dạy lại phần suy luận cho thị giác? Kiến trúc VLM vì thế rất đơn giản về ý tưởng và gồm đúng ba khối:

1. **Vision encoder** — thường là CLIP hoặc SigLIP — biến ảnh thành một chuỗi *token* thị giác.
2. **Khối kết nối** — một MLP chiếu, hoặc một tập truy vấn học được kiểu Q-Former [3] — đưa chuỗi đó về cùng không gian với *token* văn bản.
3. **LLM** — xử lý chuỗi hỗn hợp hệt như xử lý văn bản thuần [4].

Trọng tâm thiết kế nằm ở khối thứ hai, và nó là một cuộc đánh đổi kinh điển. Chiếu MLP thẳng giữ nguyên số *token* thị giác nên giữ được chi tiết không gian, nhưng làm chuỗi dài ra và chi phí *attention* tăng theo bình phương. Nén xuống một số ít *token* truy vấn thì rẻ, nhưng chính bước nén đó vứt đi thông tin định vị — và đó là lý do nhiều VLM mô tả ảnh trôi chảy nhưng chỉ sai vị trí. Cái được là VQA, *grounding*, *captioning* và điều khiển giao diện bằng một mô hình duy

nhất; cái mất là chi phí, khả năng kiểm soát, và một dạng lỗi mới: mô hình bịa ra chi tiết không có trong ảnh với đúng giọng tự tin mà phần ngôn ngữ đã học được.

Hình 1.5 đặt hai biến thể của khối kết nối cạnh nhau, vì chính chỗ này quyết định VLM của bạn định vị được hay không.



Hình 1.5. Hai cách nối bộ mã hoá thị giác vào một LLM. Số ô nhỏ giữa khối kết nối và LLM là số **token** thị giác thực sự đi vào chuỗi; đó là đại lượng duy nhất quyết định cả chi phí lẫn khả năng định vị, nên đây là lựa chọn thiết kế quan trọng nhất của một VLM.

F. Kết quả tích lũy – mô hình nền tảng như một thành phần

Điều đáng ngạc nhiên nhất của cả mục không phải là mô hình nền tảng mạnh, mà là nó mạnh ở *những nơi lẽ ra nó không nên mạnh*. Lấy đặc trưng DINOv2 hoặc DINOv3 đóng băng rồi gắn thêm một đầu tuyến tính, bạn thường thắng một mạng train từ đầu ngay trên dữ liệu chuyên ngành [5]. Điều đó đúng với ảnh nội soi, ảnh vệ tinh và ảnh vi thể, ở mức vài trăm tới vài nghìn nhãn. Trực giác thông thường nói dữ liệu trong miền phải thắng dữ liệu ngoài miền; thực nghiệm nói vài trăm mẫu không đủ để học lại những thống kê ảnh cơ bản mà 140 triệu ảnh đã học hộ.

Điều này thay đổi vị trí của bạn trong hệ thống: bạn không còn là người huấn luyện mô hình mà là người *tích hợp* nó — giống hệt việc dùng một solver phi tuyến có sẵn thay vì tự viết Levenberg–Marquardt. Và cũng giống hệt như vậy, trách nhiệm chuyển từ “mô hình có đúng không” sang “tôi có biết mô hình này giả định gì và hỏng ở đâu không”.

G. Giới hạn và trường hợp hỏng

- **“Mở trọng số” không đồng nghĩa với “mở”.** Một số *backbone* tự giám sát mạnh nhất phát hành kèm giấy phép riêng hạn chế dùng thương mại, và dữ liệu huấn luyện thì hầu như không bao giờ được công bố. Với sản phẩm, đây là ràng buộc cứng cần kiểm tra *trước* khi chạy thí nghiệm.
- **Chuyển giao thất bại có hệ thống khi vật lý tạo ảnh khác nguồn** — ảnh nhiệt, radar, ảnh sự kiện, đa phổ. Ở đó thống kê tần số, phân bố nhiễu và cả số kênh đều khác, nên giả định “cạnh vẫn là cạnh” không còn đúng (xem A/02-vat-ly-tao-anh/04-cam-bien-khac-va-domain-gap).
- **Tự giám sát không miễn nhiễm với thiên lệch dữ liệu; nó chỉ làm thiên lệch khó nhìn hơn,** vì không còn bảng phân bố nhãn để kiểm tra. Nếu 90% ảnh không nhãn của bạn chụp ban ngày, biểu diễn học ra sẽ tốt ban ngày và không ai cảnh báo bạn.
- **Đánh giá bằng *linear probe* che giấu rất nhiều.** Một biểu diễn có thể tuyến tính phân tách tốt các lớp phổ biến mà vẫn vô dụng cho định vị chính xác — đúng loại thất bại quan trọng nhất nếu đầu ra của bạn là hộp, *mask* hay tương ứng điểm.

CẠM BẤY

Bất biến mà bạn ép vào *augmentation* chính là thông tin bạn cố tình vứt đi. *Color jitter* mạnh dạy mô hình rằng màu không quan trọng — một thảm hoạ nếu tác vụ đích là phát

hiện gỉ sét, phân loại nhuộm mô bệnh học, hay đọc đèn tín hiệu. *Random resized crop* dạy mô hình rằng tỉ lệ không quan trọng — một thảm họa nếu bạn cần ước lượng kích thước thật. Trước khi sao chép một công thức *augmentation* từ paper tự giám sát, hãy liệt kê các bất biến nó áp đặt và đối chiếu từng cái với thứ tác vụ của bạn cần *phân biệt*.

CẦN NHỚ

Mỗi bước trong chuỗi này đổi *nhân đất và chính xác lấy tín hiệu rẻ và gián tiếp*. Nó gần như luôn có lỗi, nhưng cái bạn mất là khả năng biết mô hình đã học gì — và bạn chỉ phát hiện ra điều đó ở đúng những trường hợp biên mà bạn quan tâm nhất. Hệ quả hành động: với mọi mô hình nền tảng, hãy tự thu một tập đánh giá nhỏ trên đúng phân phối triển khai trước khi tin bất kỳ con số nào. [T2]

H. Kết nối

Mục này là tiền đề trực tiếp của C/14-bai-toan-cv-loi: phát hiện từ vụng mở chỉ là bộ đề xuất vùng cấm vào không gian *embedding* CLIP. Nó đối lập với B/06-chat-luong-du-lieu. Chương kia lập luận rằng chất lượng nhãn quyết định trần hiệu năng; mục này lập luận rằng phần lớn tín hiệu lấy được mà không cần nhãn. Cả hai đều đúng, và ranh giới nằm ở chỗ bạn cần *ngữ nghĩa* hay cần *biểu diễn*. Về phía sau, C/18/02-kinh-te-hoc-pretraining trả lời câu hỏi định lượng: đầu tư compute vào đâu, và khi nào tinh chỉnh đáng giá hơn đóng băng.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao BYOL không sụp về nghiệm hằng số dù không có một *negative* nào, và bỏ thành phần nào thì nó sụp?
2. Với 500 ảnh nội soi, *linear probe* trên DINOv3 hay *fine-tune* toàn bộ ResNet ImageNet sẽ thắng, và lập luận đó dựa trên đại lượng nào — “số tham số được phép thay đổi so với lượng thông tin mới” nói gì ở đây?
3. Hạ nhiệt độ τ trong InfoNCE làm gradient dịch chuyển đi đâu, và tác dụng phụ nào xuất hiện khi tập dữ liệu có nhiều ảnh gần trùng nhau?
4. MAE cho *linear probe* kém hơn DINO nhưng tinh chỉnh lại tốt hơn. Điều đó nói gì về loại thông tin mà hai mục tiêu huấn luyện giữ lại?
5. Vì sao “mở trọng số” không đồng nghĩa với “mở”, và điều đó ảnh hưởng gì tới lựa chọn *backbone* cho một sản phẩm thương mại?
6. Tác vụ nào sẽ bị công thức *augmentation* chuẩn của SimCLR làm hỏng biểu diễn, và bất biến nào bị ép sai ở đó?

Nguồn

- [1] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [2] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [3] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [4] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [5] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [6] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [7] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [8] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [9] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [10] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [11] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [12] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [13] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [14] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [15] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [16] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [17] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [18] N. Houlsby, A. Giurghi, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.
- [19] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [20] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

1.2 Kinh tế học của pretraining (The Economics of Pretraining)

PHỤ THUỘC C/18/01-nguồn-giam-sat (mục ngay trước, bắt buộc); B/11-gioi-han-tinh-toan (compute, băng thông, FLOPs); B/07-chia-du-lieu-va-danh-gia (vì sao nhiệm vụ tập kiểm tra phá huỷ mọi kết luận); E/24-thi-nghiem-va-ablation (cách chạy thí nghiệm điểm giao cho từ từ).

TÓM TẮT

Mục này là mặt định lượng của việc tiền huấn luyện: có một ngân sách compute cố định thì chia cho tham số và dữ liệu theo tỉ lệ nào, và khi nào tính chính đáng giá hơn đồng bằng. Đọc xong bạn ước lượng được chi phí một lần tiền huấn luyện bằng một phép nhân trên giấy. Bạn đọc được một scaling law mà không bị nó lừa. Và bạn thiết kế được thí nghiệm điểm giao cho chính dữ liệu của mình, thay vì tin lời khuyên chung. Nếu chỉ nhớ một điều: compute, tham số và dữ liệu là ba mặt của cùng một ngân sách — bài học Chinchilla là ví dụ đắt nhất lịch sử ngành về việc tối ưu một mặt mà quên hai mặt kia.

KIẾN THỨC CẦN CÓ TRƯỚC

- **FLOPs và ngân sách compute** — FLOPs là số phép tính dấu phẩy động. Ngân sách compute C là tổng FLOPs bạn mua được cho một lần huấn luyện; nó cố định, nên chọn mô hình to hơn đồng nghĩa với chọn dữ liệu ít hơn.
- **token** — đơn vị nhỏ nhất của dữ liệu huấn luyện mà mô hình xử lý — một mẫu từ với văn bản, một mảnh ảnh (*patch*) với ViT. Lượng dữ liệu D luôn được đếm bằng token, không phải bằng số file.
- **hàm mất mát (loss)** — con số đo mức sai của mô hình trên dữ liệu; huấn luyện là quá trình giảm nó. Mọi “scaling law” trong mục này là công thức dự đoán con số đó sẽ giảm tới đâu.
- **scaling law** — quan hệ định lượng khớp từ thực nghiệm giữa loss, số tham số, lượng dữ liệu và compute. Giá trị của nó là cho phép *ngoại suy*: chạy vài mô hình nhỏ rồi dự đoán mô hình lớn trước khi tiêu tiền.
- **tập kiểm tra (test set) và tập validation** — tập validation dùng để chọn siêu tham số; tập test dùng đúng một lần để báo cáo. Trộn hai vai trò này là cách phổ biến nhất để tự lừa mình.
- **siêu tham số (hyperparameter)** — tham số bạn chọn chứ không phải mô hình học, ví dụ *learning rate* (độ dài mỗi bước cập nhật trọng số), kích thước *batch*, số *epoch*.
- **checkpoint** — ảnh chụp toàn bộ trọng số mô hình (và thường cả trạng thái optimizer) tại một thời điểm huấn luyện, lưu ra đĩa để chạy tiếp hoặc để triển khai.
- **seed** — số khởi tạo bộ sinh ngẫu nhiên. Đổi seed là đổi khởi tạo trọng số, thứ tự dữ liệu và augmentation — nên hai lần chạy khác seed cho hai kết quả khác nhau ngay cả khi mã hoàn toàn giống nhau.

A. Đặt vấn đề

Mục trước trả lời câu hỏi *lấy giám sát ở đâu*. Mục này trả lời câu hỏi đắt tiền hơn: *có một ngân sách compute cố định thì tiêu vào đâu?* Đây không phải câu hỏi triết học — nó là câu hỏi phân bổ nguồn lực, và trong nhiều năm cả ngành đã trả lời sai nó theo cùng một hướng.

Vấn đề khó ở chỗ ba đại lượng — số tham số N , lượng dữ liệu D , và compute C — ràng buộc nhau. Thêm tham số mà không thêm dữ liệu thì mô hình có thêm chỗ chứa mà không có gì để chứa. Thêm dữ liệu mà không thêm tham số thì mô hình bão hoà. Và cả hai đều bị chặn bởi tổng số phép tính bạn mua được. Nếu chỉ có trực giác, cách duy nhất tìm điểm cân bằng là thử; nhưng mỗi lần thử ở quy mô nghìn tỉ tham số tốn hàng triệu đô la, nên thử mù không chấp nhận được. Điều ngành cần là một *luật cho phép ngoại suy*: chạy vài mô hình nhỏ, khớp một đường cong, rồi dự đoán mô hình lớn trước khi tiêu tiền.

Học xong mục này thì làm được gì? Ba việc mà trước đó bạn chỉ làm được bằng cách hỏi người khác. Một, tính chi phí huấn luyện của một mô hình bất kỳ bằng một phép nhân trên giấy. Hai, phát hiện khi một bảng kết quả đang so hai mô hình có ngân sách khác nhau. Ba, thiết kế thí nghiệm điểm giao trả lời đúng câu hỏi bạn đang có.

TRỰC GIÁC

Hãy mượn ngôn ngữ của **hiệu chỉnh chùm tia** (*bundle adjustment*), với ý thức rằng đây là phép loại suy chứ không phải định lý. Trong BA, số ảnh là toạ độ điểm và tư thế camera; số phương trình là số quan sát. Nếu bạn thêm điểm 3D mà không thêm quan sát, hệ trở nên thiếu ràng buộc: nghiệm vẫn tồn tại, sai số tái chiếu vẫn nhỏ, nhưng cấu trúc thu được là ảo — nó khớp quan sát mà không tương ứng với thế giới.

Bài học Chinchilla nói rằng cả ngành đã chạy đúng tình huống đó suốt vài năm: mô hình được thổi phồng số tham số trong khi số “quan sát” — *token* dữ liệu — gần như đứng yên. Loss huấn luyện vẫn giảm, bảng kết quả vẫn đẹp, nên không ai thấy có gì sai. Chỗ mà phép loại suy gây cũng đáng nhớ. Mạng nơ-ron quá tham số hoá vẫn tổng quát được, nên “thiếu ràng buộc” ở đây không sinh nghiệm vô nghĩa như trong BA. Nó chỉ sinh ra *lãng phí*: cùng số FLOPs ấy lẽ ra mua được mô hình tốt hơn.

B. Scaling laws — từ thử mù sang dự đoán (Scaling Laws)

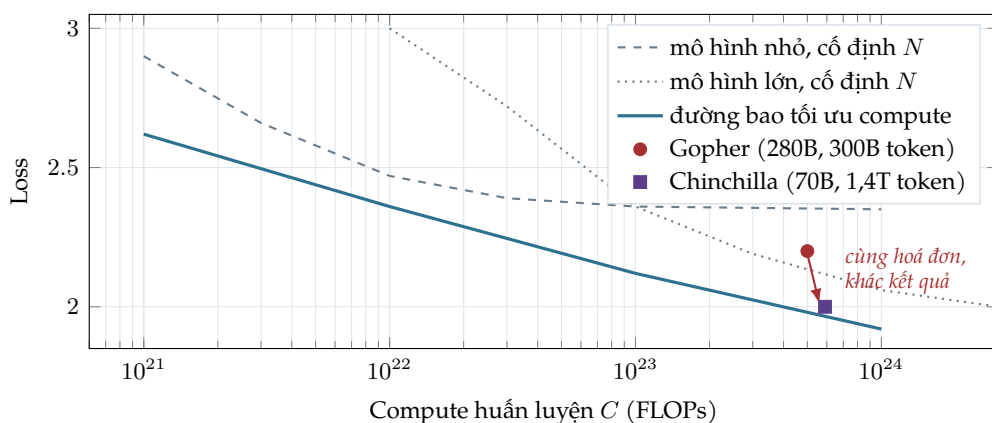
Quan sát thực nghiệm nền tảng là: khi tăng N , D và C , loss giảm theo quan hệ lũy thừa (*power law*) chứ không theo bậc thang [13]. Dạng thường dùng là

$$L(N, D) = L_{\infty} + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}.$$

Công thức này thực sự nói điều gì? Rằng loss của bạn là tổng của ba thứ độc lập. Thứ nhất là phần sàn không bao giờ giảm được, L_{∞} : entropy thật của dữ liệu, cộng mọi thứ mà kiến trúc này về nguyên tắc không biểu diễn nổi. Thứ hai là phần phạt vì mô hình quá nhỏ. Thứ ba là phần phạt vì dữ liệu quá ít. Muốn giảm loss, bạn phải biết mình đang bị phạt ở số hạng nào — và đó chính là điều mà đường cong cho bạn biết.

Điều làm công thức có giá trị thực dụng không phải hình dạng của nó, mà là việc nó *khớp được từ các mô hình nhỏ và ngoại suy đúng lên vài bậc độ lớn*. Đó là lần đầu tiên trong lịch sử ngành, một quyết định chi tiêu chín chữ số được đưa ra dựa trên đường cong khớp từ thí nghiệm rẻ tiền. Hiện tượng tương tự đã được xác nhận cho thị giác với ViT [14], dù các hằng số hoàn toàn khác.

Hình 1.6 cho thấy vì sao lập luận này không nhìn ra được nếu chỉ theo dõi một mô hình: mỗi kích thước mô hình có đường loss riêng, và điều đáng quan tâm là *đường bao* của cả họ chứ không phải bất kỳ đường nào trong đó.



Hình 1.6. Đường bao tối ưu compute và hai mô hình cùng ngân sách (*trục và hình dạng minh hoạ; chỉ vị trí tương đối của Gopher với Chinchilla là dựa trên kết quả đã công bố*). Một mô hình cố định kích thước sẽ rời khỏi đường bao khi ta đổ thêm dữ liệu vào nó: loss chững lại vì nó đã hết chỗ chứa. Gopher nằm phía trên đường bao ở đúng mức compute của Chinchilla — đó là hình ảnh của việc thừa tham số và thiếu dữ liệu.

↔ **đối lập với** B/11 — chương kia đo giới hạn vật lý của phần cứng, mục này đo giới hạn kinh tế của việc dùng phần cứng đó — cùng một ngân sách nhìn từ hai phía.

Hai hệ quả cần đọc kỹ:

- **Lợi ích giảm dần nhưng không bao giờ dừng.** Luật lũy thừa không có ngưỡng mà thêm dữ liệu trở nên vô ích — chỉ có ngưỡng mà nó không còn đáng tiền. Câu hỏi đúng luôn là “đáng bao nhiêu”, không phải “còn tác dụng không”.
- **Scaling không sửa được lỗi bài toán.** Số hạng L_∞ nói rằng nếu nhân bản hoặc bài toán bị đặt sai, mọi đường cong đều chạm sàn ở một chỗ cao và không lượng compute nào cứu được.

C. Bài học Chinchilla (The Chinchilla Lesson)

Câu hỏi tiếp theo: với C cho trước, chọn N và D thế nào? Xấp xỉ chuẩn cho **transformer** là

$$C \approx 6ND \quad (\text{FLOPs cho toàn bộ quá trình huấn luyện}),$$

với hệ số 6 đến từ hai FLOPs cho mỗi phép nhân–cộng, nhân với ba lượt (một **lượt xuôi** (*forward pass*), hai **lượt ngược** (*backward pass*)). Diễn giải bằng lời: chi phí huấn luyện tỉ lệ thuận với tích của kích thước mô hình và lượng dữ liệu — nên với ngân sách cố định, hai đại lượng ấy nằm trên một đường hyperbol và bạn chỉ được chọn một điểm trên đó. Đáp số thực nghiệm là N và D nên tăng **cùng tốc độ**, mỗi cái tỉ lệ với $\sqrt{C}$ [15].

Ví dụ nhỏ nhất, dùng hai mô hình đã công bố.

- **Gopher:** $N = 280$ tỉ tham số, $D = 300$ tỉ **token** $\Rightarrow C = 6 \times 2,8 \times 10^{11} \times 3 \times 10^{11} \approx 5,0 \times 10^{23}$ FLOPs. Tỉ lệ $D/N = 1,07$.
- **Chinchilla:** $N = 70$ tỉ, $D = 1400$ tỉ **token** $\Rightarrow C = 6 \times 7 \times 10^{10} \times 1,4 \times 10^{12} \approx 5,9 \times 10^{23}$ FLOPs. Tỉ lệ $D/N = 20$.

Cùng bậc compute, tức cùng hoá đơn. Mô hình nhỏ hơn bốn lần nhưng ăn nhiều dữ liệu hơn gần năm lần thẳng rõ rệt trên hầu hết bài kiểm tra. Con số 20 **token** cho mỗi tham số chính là quy tắc ngón tay cái người ta nhớ.

Cơ chế của cái sai đáng học hơn con số 20. Số tham số là đại lượng dễ công bố, dễ so sánh và dễ gây ấn tượng; lượng **token** thì không ai khoe. Khi một đại lượng dễ đo trở thành đại lượng được tối ưu, nó thôi là đại lượng đúng — cùng một cơ chế đã làm hỏng nhiều bảng xếp hạng ở

E/24-thi-nghiem-va-ablation. Lưu ý thêm: con số 20 chỉ tối ưu cho *chi phí huấn luyện*. Mô hình phục vụ hàng tỉ lượt **suy luận** (*inference*) thì hoá đơn nằm ở phía chạy chứ không phải phía huấn luyện. Tối ưu tổng chi phí vòng đời vì thế đẩy bạn về mô hình *nhỏ hơn nữa*, huấn luyện lâu hơn nữa. Đó là lý do các mô hình mở gần đây thường vượt xa tỉ lệ 20.

D. Chất lượng hơn số lượng (Data Quality over Quantity)

Bước tiếp theo của cùng lập luận: nếu D quan trọng đến thế thì D là gì càng quan trọng hơn. Một dữ liệu bị lặp mười lần không mang thêm thông tin nào nhưng vẫn ngốn đúng số FLOPs như dữ liệu mới. Với thị giác, hiệu ứng còn mạnh hơn vì ảnh web trùng lặp ở mức độ khủng khiếp: cùng một logo, cùng một ảnh *stock*, cùng một khung hình phim xuất hiện hàng nghìn lần.

Điều này gắn thẳng vào B/06-chat-luong-du-lieu: các tập dữ liệu chuẩn của ngành có tỉ lệ nhãn sai không nhỏ, đủ để đảo thứ hạng giữa các mô hình gần nhau [16]. Khi bạn tự tuyển *corpus*, một giờ viết bộ lọc trùng lặp thường có lãi cao hơn một giờ GPU.

E. Nhiễm bẩn tập kiểm tra — vấn đề do chính pretraining sinh ra

Tiền huấn luyện quy mô web tạo ra một vấn đề mà các phương pháp cũ không có: **nhiễm bẩn tập kiểm tra** (*test set contamination*). Nếu ảnh hoặc câu hỏi trong tập test đã nằm đâu đó trong *corpus* tiền huấn luyện, con số đánh giá không đo khả năng tổng quát hoá mà đo khả năng ghi nhớ. Cái làm nó nguy hiểm không phải bản thân hiện tượng — ai cũng biết là xấu — mà là việc nó *không kiểm chứng được*: *corpus* thường không công bố, và ngay cả khi công bố thì đối sánh hàng tỉ ảnh với tập test cũng không rẻ.

Có một dạng tinh vi hơn và phổ biến hơn: nhiễm bẩn qua cộng đồng. Một tập test công khai được cả ngành dùng mười năm sẽ bị việc chọn siêu tham số bào mòn dần, ngay cả khi không ai nhìn trực tiếp vào nó. Khi người ta dựng lại một tập test mới theo đúng quy trình cũ, hiệu năng mọi mô hình đều tụt, và tụt gần như song song [17]. Tính song song đó là tin tốt — thứ hạng tương đối còn dùng được — nhưng giá trị tuyệt đối thì không.

Kết luận thực dụng: với bất kỳ mô hình nền tảng nào, con số *zero-shot* trên *benchmark* công khai nên coi là chặn trên lạc quan. Bằng chứng duy nhất đáng tin là tập đánh giá do chính bạn thu, *sau* ngày mô hình được huấn luyện, trên đúng phân phối bạn sẽ triển khai.

→ **tiền đề** B/07 — quy trình chia dữ liệu sạch là điều kiện để con số đánh giá có nghĩa; nhiễm bẩn chỉ là rò rỉ dữ liệu ở quy mô web, nơi bạn không kiểm toán được nguồn.

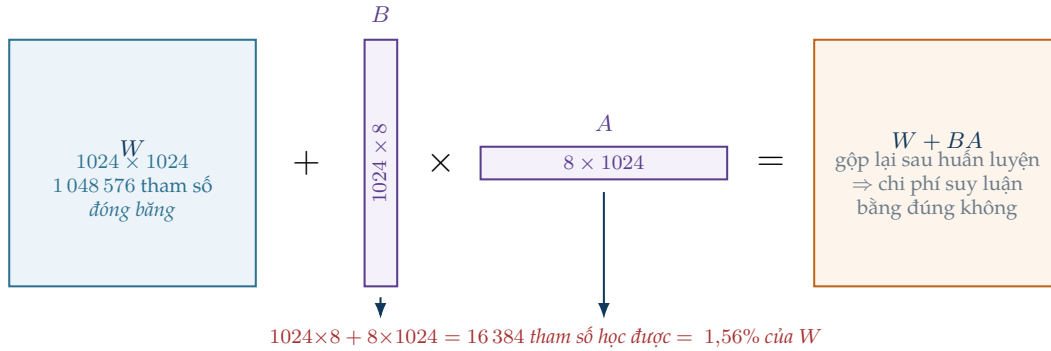
F. PEFT — đổi lại kinh tế học của việc thích nghi

Vấn đề rất cụ thể: tinh chỉnh toàn bộ một mô hình lớn nghĩa là lưu một bản sao toàn bộ trọng số cho mỗi khách hàng, mỗi tác vụ, mỗi phiên bản — và trạng thái *optimizer* của Adam còn tốn gấp đôi số đó. **PEFT** (tinh chỉnh chỉ một phần rất nhỏ tham số) giải bài toán này bằng cách đóng băng trọng số gốc và chỉ học một phần rất nhỏ tham số thêm vào:

- **Adapter** — chèn các khối nhỏ giữa các tầng, chỉ chúng được học [18].
- **LoRA** — phổ biến hơn: viết cập nhật dưới dạng tích hai ma trận hạng thấp, $\Delta W = BA$ với $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$ [19]. *Ý nghĩa*: ta giả định rằng “khoảng cách” giữa mô hình gốc và mô hình bạn cần nằm trong một không gian con rất thấp chiều, nên chỉ cần học toạ độ trong không gian con đó.
- **Prompt tuning** — không đụng trọng số nào, chỉ học một chuỗi *token* đầu vào. Rẻ nhất, cũng yếu nhất.

Con số làm rõ mọi thứ. Một ma trận trọng số 1024×1024 có 1 048 576 tham số. LoRA hạng $r = 8$ thay nó bằng $1024 \times 8 + 8 \times 1024 = 16\,384$ tham số, tức 1,56%. Nhân lên toàn mô hình: một *adapter* vài chục megabyte thay cho một *checkpoint* vài chục gigabyte — và vì ΔW cộng thẳng vào W được sau khi huấn luyện, chi phí suy luận bằng đúng không.

Hình 1.7 vẽ cùng phép tính đó dưới dạng diện tích, vì đó là cách nhanh nhất để thấy vì sao con số 1,56% không phải một thủ thuật mà là hệ quả hình học của việc chọn $r \ll d$.



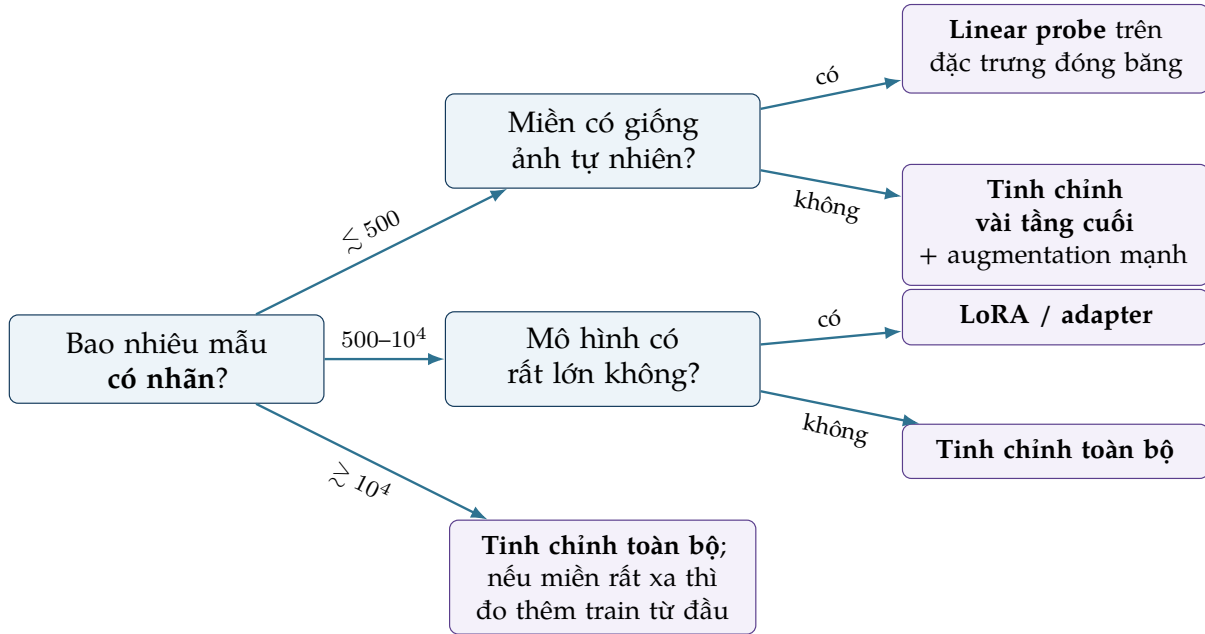
Hình 1.7. LoRA thay cập nhật đầy đủ ΔW bằng tích hai ma trận hạng thấp. Diện tích hai dải mỏng so với hình vuông lớn chính là tỉ lệ tham số phải học và phải lưu; và vì BA cộng thẳng vào W sau khi huấn luyện nên phần tiết kiệm này không phải trả lại ở lúc suy luận. Giả định ngầm là cập nhật cần thiết thật sự nằm trong một không gian con r chiều.

Giả định ngầm cần gọi tên: LoRA tin rằng *cập nhật cần thiết để thích nghi có hạng thấp*. Điều này hợp lý khi tác vụ đích là biến thể của thứ mô hình đã biết. Nó sai khi bạn đòi mô hình học một năng lực thực sự mới; khi đó hạng thấp thành cổ chai, và triệu chứng là loss huấn luyện dừng ở mức cao hơn hẳn tinh chỉnh toàn bộ bất kể học bao lâu. Phản ứng đúng lúc đó là tăng r hoặc mở thêm module, không phải tăng *learning rate*.

↔ cùng nguyên lý với $D/20$ — nén mô hình cũng giả định cái cần giữ nằm trong một không gian con nhỏ; LoRA là phiên bản của giả định đó ở phía thích nghi thay vì ở phía suy luận.

G. Quyết định cuối cùng vẫn là của bạn: thí nghiệm điểm giao

Mọi thứ ở trên là kiến thức chung. Hình 1.8 nén nó thành một cây quyết định dùng được trong ba mươi giây, và Bảng 1.3 nói rõ từng nhánh hổng ở đâu. Cả hai chỉ thu hẹp không gian tìm kiếm — chúng không kết luận thay bạn.

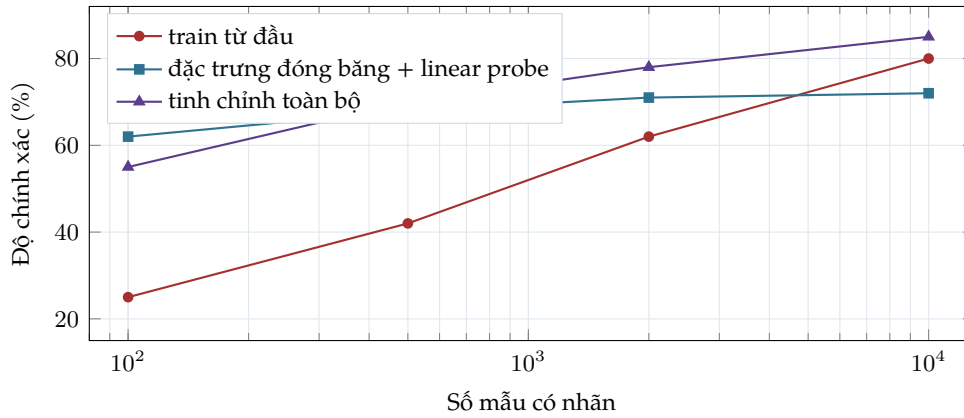


Hình 1.8. Cây quyết định chọn chế độ thích nghi. Các ngưỡng là điểm khởi đầu chứ không phải hằng số phổ quát — thí nghiệm điểm giao ở Hình 1.9 là cách xác định ngưỡng thật cho bài toán của bạn.

Bảng 1.3. Năm chế độ thích nghi, xếp theo số tham số được phép thay đổi.

Chế độ	Cơ chế	Giả định	Hỏng khi nào
Train từ đầu	Học mọi thứ từ dữ liệu của bạn	Dữ liệu đủ để học lại cả thống kê ảnh cơ bản	Dưới vài chục nghìn mẫu: gần như luôn thua
Linear probe	Chỉ học một siêu phẳng trên đặc trưng đóng băng	Thông tin cần thiết đã tuyến tính tách được	Tác vụ cần định vị chính xác, hoặc đặc trưng không có sẵn trong nguồn
Tinh chỉnh vài tầng cuối	Mở phần chuyên biệt của nguồn	Khác biệt miền nằm ở tầng cao	Khác biệt nằm ở tầng thấp (ảnh nhiệt, radar, đa phổ)
LoRA / adapter	Học cập nhật hạng thấp, gốc đóng băng	Cập nhật cần thiết có hạng thấp	Cần năng lực thực sự mới: loss chứng ở mức cao
Tinh chỉnh toàn bộ	Mở toàn bộ	Đủ dữ liệu để không quên đặc trưng gốc	Dữ liệu ít: overfit và quên thảm khốc

Thí nghiệm đáng làm nhất của cả chương là thí nghiệm điểm giao: chạy ba chế độ — train từ đầu, đóng băng, tinh chỉnh toàn bộ — ở các mức 100, 500, 2000, 10 000 mẫu, mỗi mức lặp ba *seed*, rồi vẽ ba đường cong trên một hình. Hình 1.9 minh họa hình dạng thường thấy, với cảnh báo rõ ràng rằng các con số là *minh họa*, không phải kết quả đo của một tập dữ liệu cụ thể.



Hình 1.9. Hình dạng điển hình của thí nghiệm điểm giao (số liệu minh họa, không phải kết quả đo). Đường đồng bằng bão hoà sớm nhưng thắng ở vùng dữ liệu rất ít; tinh chỉnh toàn bộ vượt lên sau điểm giao thứ nhất; train từ đầu chỉ cạnh tranh ở vùng dữ liệu lớn. Vị trí hai điểm giao khác nhau cho từng bài toán — đó chính là thứ bạn phải đo.

Hai điểm giao trên hình là hai con số đáng giá hơn mọi lời khuyên chung, vì chúng trả lời đúng câu hỏi bạn thực sự có: *với lượng nhãn tôi đang có, và với lượng nhãn tôi mua thêm được trong quý này, chế độ nào đúng?* Và vì trục hoành là log, chỉ bốn điểm đo đã đủ để nhìn ra hình dạng.

H. Giới hạn và trường hợp hỏng

- **Scaling law là quan hệ khớp thực nghiệm, không phải định luật.** Nó chỉ đúng *trong công thức huấn luyện đã dùng để khớp nó*: đổi kiến trúc, đổi lịch *learning rate*, đổi phân bố dữ liệu là các hằng số thay đổi và đường ngoại suy trượt. Đã có ít nhất một lần cả ngành ngoại suy sai vì công thức khớp có một chi tiết lịch học không nhất quán — chính là câu chuyện Chinchilla.
- **Scaling law mô tả loss, không mô tả năng lực.** Loss giảm mượt trong khi các năng lực rời rạc (đếm, suy luận nhiều bước) xuất hiện không mượt, nên đường cong đẹp không cho phép hứa hẹn gì về hành vi ở tác vụ đích.
- **LoRA dễ bị dùng để né việc chẩn đoán.** Khi kết quả kém, người ta tăng hạng thay vì hỏi liệu bài toán có thực sự là bài toán thích nghi hay không.
- **So sánh chỉ có nghĩa khi công sức tune ngang nhau.** So một *baseline* chưa tune với phương pháp đã tune là dạng thí nghiệm phổ biến nhất và vô giá trị nhất trong ngành [20].

CẠM BÃY

Ba lỗi hay đi cùng nhau khi chạy thí nghiệm điểm giao. **Một:** dùng cùng một tập *validation* để chọn siêu tham số cho cả ba chế độ rồi báo cáo trên chính tập đó — điểm giao dịch chuyển theo hướng có lợi cho chế độ được tune nhiều nhất. **Hai:** chạy một *seed* mỗi điểm; với 100 mẫu, độ lệch giữa các *seed* thường lớn hơn khoảng cách giữa hai chế độ, nên đường cong bạn vẽ là nhiễu. **Ba:** quên rằng *backbone* đã tiền huấn luyện có thể đã nhìn thấy dữ liệu đích của bạn — nếu ảnh của bạn từng ở trên internet, “*zero-shot*” không còn là *zero-shot*.

CẦN NHỚ

Compute, tham số và dữ liệu là ba mặt của cùng một ngân sách; tối ưu một mặt mà không nhìn hai mặt kia là cách sai tiền có hệ thống. Nhớ kèm hai công thức: $C \approx 6ND$ để ước lượng hoá đơn, và tỉ lệ $D/N \approx 20$ như điểm khởi đầu (không phải chân lý). Và mọi quy

tắc chung chỉ là điểm khởi đầu — con số duy nhất ràng buộc quyết định của bạn là điểm giao đo trên chính dữ liệu của bạn. [T2]

I. Kết nối

Mục này là mặt định lượng của *C/18/01-nguồn-giam-sat*: mục kia nói giám sát lấy ở đâu, mục này nói mua bao nhiêu thì đủ. Nó là tiền đề của *D/20-toi-uu-va-nen-mo-hinh*, vì phần lớn kỹ thuật nén chỉ là cách mua lại compute suy luận bằng compute huấn luyện — cùng một logic ngân sách, chỉ khác cột chi. Nó phụ thuộc rất chặt vào *B/07-chia-du-lieu-va-danh-gia*: không có quy trình chia dữ liệu sạch thì nhiệm bản biến mọi con số ở đây thành vô nghĩa. Và nó đối lập có ích với *B/11-gioi-han-tinh-toan*: chương kia đo giới hạn vật lý của phần cứng, mục này đo giới hạn kinh tế của việc dùng phần cứng đó.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao Gopher và Chinchilla tốn xấp xỉ cùng số FLOPs, và điều gì trong cách ngành đo lường tiến bộ đã khiến sai lầm này sống sót nhiều năm?
2. Tỷ lệ 20 *token* trên tham số tối ưu cho chi phí gì? Nếu mô hình sẽ phục vụ một tỉ lượt suy luận thì tỉ lệ đúng dịch về phía nào, và vì sao?
3. Số hạng L_∞ nói gì về việc dùng compute để sửa một bài toán bị đặt sai?
4. LoRA hạng 8 trên ma trận 1024×1024 tiết kiệm bao nhiêu phần trăm tham số, và triệu chứng nào cho biết giả định hạng thấp đã bị vi phạm?
5. Vì sao khi dựng lại một tập test theo đúng quy trình cũ thì mọi mô hình đều tụt điểm, và vì sao việc chúng tụt gần như song song lại là tin tốt?
6. Ba cách phổ biến làm cho thí nghiệm điểm giao ra kết quả sai là gì, và sửa từng cái thế nào?

Nguồn

- [1] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [2] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [3] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [4] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [5] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [6] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [7] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [8] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [9] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [10] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [11] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [12] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [13] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [14] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [15] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [16] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [17] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [18] N. Houlsby, A. Giurghi, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.
- [19] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [20] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.

1.3 Thực hành

Phần thực hành của chương này có một đặc điểm khác các chương trước: bạn gần như không huấn luyện gì từ đầu. Công việc chủ yếu là chọn đúng thành phần dựng sẵn, đo xem nó hợp với dữ liệu của mình tới đâu, và biết dừng lại ở chế độ thích nghi rẻ nhất còn đủ tốt.

Repo và tài nguyên. *Backbone tự giám sát:* [facebookresearch/dinov3](#) (có cả bản cho ảnh vệ tinh; giấy phép riêng — dùng DINOv2 nếu bắt buộc Apache 2.0), [facebookresearch/mae](#), [lightly-ai/lightly](#) và [lightly-train](#) khi muốn tự tiền huấn luyện trên dữ liệu của chính mình. *Thị giác-ngôn ngữ:* [mlfoundations/open_clip](#), SigLIP. *VLM mở đáng thử tại thời điểm viết:* Qwen3-VL (mạnh toàn diện, OCR và video tốt), InternVL3 (giấy phép MIT, thoáng nhất nhóm dẫn đầu), Molmo2 của AI2 (mở cả dữ liệu huấn luyện — lựa chọn đúng khi cần tái lập), Pixtral (Apache 2.0). *Thích nghi:* [huggingface/peft](#). Danh sách tên mô hình này chốt tại tháng 9/2026 và sẽ cũ trong 6–12 tháng, trong khi các repo hạ tầng như [peft](#) hay [timm](#) bền hơn nhiều — hãy nhớ tên hạ tầng, tra lại tên mô hình.

Câu hỏi kiểm tra hiểu. Với 500 ảnh trong một lĩnh vực rất hẹp, chẳng hạn ảnh nội soi, thì *linear probe* trên DINOv3 hay tinh chỉnh toàn bộ một ResNet ImageNet sẽ thắng — và lập luận của bạn dựa trên đại lượng nào? Vì sao “mở trọng số” không đồng nghĩa với “mở”, và điều đó ràng buộc thế nào tới việc đưa mô hình vào sản phẩm? Nếu phải giải thích cho một đồng nghiệp trong ba câu vì sao BYOL không sụp dù không có *negative*, ba câu đó là gì?

Mini project (vài giờ đến hai ngày). Chạy thí nghiệm điểm giao trên một bài toán phân loại nhỏ của riêng bạn: ba chế độ train từ đầu, đóng băng và tinh chỉnh toàn bộ, ở bốn mức 100, 500, 2000 và 10 000 mẫu, mỗi ô ba *seed*. Vẽ ba đường cong trên một hình kèm dải phương sai giữa các *seed*. Sản phẩm cuối không phải cái hình, mà là hai con số: hai hoành độ điểm giao — ngưỡng quyết định cho chính bài toán của bạn.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Học chuyển giao	Lấy trọng số đã học trên một tập lớn làm điểm khởi đầu thay vì khởi tạo ngẫu nhiên. Đáng thử đầu tiên; hóng khi vật lý tạo ảnh của miền đích khác nguồn, ví dụ ảnh nhiệt hay radar.
●●● Linear probe	Đóng băng backbone, chỉ huấn luyện một tầng tuyến tính trên đặc trưng. Phép thử rẻ nhất xem đặc trưng có chứa thông tin cần thiết; che giấu việc mô hình định vị kém.
●● Vì sao mở nhiều tham số lại hại khi ít dữ liệu?	Số tham số được phép thay đổi phải tương xứng với lượng thông tin mới. Vài trăm mẫu không trả nổi cho cả mạng, nên bạn nhận overfitting và quên thảm khốc.
●●● Học tự giám sát	Tạo bài toán phụ mà đáp án suy ra từ chính bức ảnh, nên không cần nhãn người. Cái giá là bài toán phụ có nghiệm tầm thường phải chống.
●●● Sụp biểu diễn	Mô hình xuất ra cùng một vector cho mọi ảnh, thoả mọi ràng buộc “giống nhau” mà không học gì. Chống nó là toàn bộ nội dung kỹ thuật của tự giám sát.
●● Vì sao contrastive cần rất nhiều negative?	Lực đẩy từ negative chính là cơ chế chống sụp duy nhất của nó. Batch nhỏ thì tín hiệu đẩy yếu; dữ liệu ít lớp thì negative hoá ra là positive.
●● Masked modeling	Che một phần ảnh rồi bắt mô hình khôi phục. Không cần chống sụp, nhưng đặc trưng thô nên linear probe kém — phải fine-tune mới mạnh.
●● Hạ nhiệt độ trong InfoNCE dồn gradient đi đâu?	Về những cặp âm có độ tương đồng gần với cặp dương. Vặn quá thấp thì chỉ học từ vài cặp khó nhất và rất nhạy với ảnh gần trùng nhau.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● CLIP	Tương phản giữa ảnh và chú thích của chính nó, trên ma trận $N \times N$ trong một batch. Cho tập nhân mở và zero-shot; yếu ở quan hệ không gian, đếm và phủ định.
●● Vì sao VLM mô tả ảnh trôi chảy nhưng sai vị trí?	Vì khối kết nối nên nhiều token thị giác xuống ít token truy vấn. Bước nén đó rẻ nhưng vứt mất thông tin định vị.
●●● Mô hình nền tảng	Mô hình lớn tiền huấn luyện trên dữ liệu quy mô rất lớn, dùng lại được cho nhiều bài toán. Thường thắng cả mạng train từ đầu ngay trong lĩnh vực hẹp.
●● Vì sao chi phí huấn luyện xấp xỉ $6ND$?	Hai FLOPs cho mỗi phép nhân-cộng, nhân ba lượt: một lượt xuôi và hai lượt ngược. Hệ số 6 chỉ đúng cho transformer huấn luyện chuẩn.
●●● Scaling law	Quan hệ lũy thừa khớp từ thực nghiệm giữa loss, tham số, dữ liệu và compute. Cho phép ngoại suy từ thí nghiệm rẻ; trượt ngay khi bạn đổi kiến trúc hoặc lịch learning rate.
●●● Chinchilla	Kết quả cho thấy phần lớn mô hình lớn thiếu dữ liệu chứ không thiếu tham số; quy tắc ngón tay cái là 20 token cho mỗi tham số. Chỉ tối ưu cho chi phí huấn luyện.
●●● Contamination	Dữ liệu test lọt vào corpus tiền huấn luyện, nên điểm số đo trí nhớ chứ không đo tổng quát hoá. Không kiểm chứng được, vì corpus thường không công bố.
●●● LoRA	Học cập nhật hạng thấp $\Delta W = BA$, gốc đóng băng; hạng 8 trên ma trận 1024×1024 chỉ tốn 1,56% tham số. Cộng thẳng vào W sau huấn luyện nên suy luận không đắt thêm.
●● Loss huấn luyện chứng cao hơn hẳn full fine-tune dù học rất lâu?	Giả định hạng thấp của LoRA đã bị vi phạm: bạn đang đòi một năng lực mới chứ không phải thích nghi. Tăng r hoặc mở thêm module, đừng tăng learning rate.
●● Zero-shot đẹp trên benchmark nhưng tệ trên dữ liệu của bạn?	Nghi contamination trước tiên, rồi nghi khoảng cách miền. Cách duy nhất kết luận được là một tập đánh giá bạn tự thu sau ngày mô hình ra đời.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
DINOv2 / DINOv3	Tự chưng cất không cần negative, dùng đúng cơ chế bất đối xứng teacher-student ở mục 18.1; đặc trưng đồng bằng cho linear probe	Hạ tầng đặc trưng phổ biến nhất cho thị giác 2024–2026; DINOv3 có bản riêng cho ảnh vệ tinh
OpenCLIP / SigLIP	Tương phản chéo phương thức ảnh-văn bản, đúng ý tưởng “ngôn ngữ là giám sát ngữ nghĩa rẻ nhất”	Bộ mã hoá được dùng lại trong gần như mọi VLM mở hiện nay
LLaVA, Qwen-VL, InternVL	Khối kết nối nối bộ mã hoá thị giác vào LLM — chính cuộc đánh đổi số token và thông tin định vị ở mục 18.1	Kiến trúc ba khối này ổn định từ 2023 tới nay dù tên mô hình đổi liên tục
huggingface/peft	LoRA và adapter, tức giả định cập nhật thích nghi có hạng thấp	Hạ tầng; là cách mặc định để tinh chỉnh mô hình lớn từ 2023
Mô hình mở gần đây	Tỉ lệ dữ liệu trên tham số vượt xa 20, đúng như lập luận tối ưu <i>tổng chi phí vòng đời</i> chứ không chỉ chi phí huấn luyện	Được cho là động cơ chính; tôi không kiểm chứng được con số cụ thể của từng mô hình

MINI PROJECT SLAM · Điểm giao trên mảnh ảnh quanh đặc trưng EuRoC · 1-2 ngày

Mục tiêu	trả lời bằng số, trên chính dữ liệu của bạn, câu hỏi “bao nhiêu mẫu thì tinh chỉnh vượt đặc trưng đóng băng”. Bảng bốn ô chỉ thu hẹp không gian; con số này mới quyết định.
Dữ liệu	EuRoC MH_01 và V1_02 để dựng tập, V2_03 giữ riêng để đánh giá. Cắt mảnh 32×32 quanh các đặc trưng ORB do chính frontend của bạn phát hiện; nhấn “cùng điểm bản đồ / khác điểm” lấy miễn phí từ liên kết dữ liệu trong bản đồ sau khi chạy hệ đầy đủ, không cần ai gán tay.
Các bước	dựng tập ở bốn mức 100, 500, 2000 và 10 000 cặp. Chạy ba chế độ trên cùng bốn mức: một CNN nhỏ huấn luyện từ đầu; <i>linear probe</i> trên DINOv2 đóng băng; tinh chỉnh toàn bộ chính <i>backbone</i> đó. Ba <i>seed</i> mỗi ô, tổng 36 lần chạy ngắn. Đo độ chính xác ghép cặp trên chuỗi giữ riêng.
Xong khi	bạn vẽ được ba đường trên một hình kèm dải phương sai giữa các <i>seed</i> , và ghi ra được <i>hai con số</i> : hai hoành độ nơi thứ hạng đảo chiều. Nếu dải phương sai chồng lên nhau tại điểm giao, kết luận đúng là “chưa phân giải được” và bạn nói được cần bao nhiêu <i>seed</i> nữa.
Nối với	C/18 cho bảng bốn ô và hình điểm giao; B/07 cho lý do phải giữ riêng cả một chuỗi chứ không chỉ giữ riêng vài khung hình. Tập mảnh ảnh dựng ở đây được dùng lại nguyên vẹn ở mini project chương 19 và 20.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Với 500 ảnh, *linear probe* trên đặc trưng nền tảng thường thắng, vì mở toàn bộ tham số của một ResNet với vài trăm mẫu là mời gọi *overfitting* và quên thăm khóc. Ở 50 000 mẫu chiều đảo lại: lúc đó dữ liệu đủ để *trả tiền* cho số tham số được mở, nên tinh chỉnh toàn bộ vượt lên — đó chính là điểm giao thứ nhất ở Hình 1.9.
- A2. Vì nghiệm hằng số bị chặn bằng *cấu trúc*, không bằng hàm mất mát: tương phản thêm lực đẩy từ negative, tự chưng cất dựng bất đối xứng teacher EMA cộng stop-gradient, còn masked modeling đặt mục tiêu khôi phục pixel vốn không có nghiệm hằng. Bỏ đúng thành phần chống sụp của mỗi họ là mô hình sụp thật.
- A3. Vì nhãn ImageNet là hàm vào một tập 1000 phần tử cố định, còn chú thích là hàm vào một không gian ngữ nghĩa mở. Chú thích nhiều hơn theo từng mẫu, nhưng nó phủ được vô hạn khái niệm và đi kèm quan hệ giữa các khái niệm — thứ mà một tập nhãn đóng không bao giờ có.
- A4. Số tham số. Đó là đại lượng dễ công bố và dễ gây ấn tượng, nên nó bị tối ưu thay cho đại lượng đúng; lượng token thì không ai khoe. Với cùng $C \approx 6ND$, Chinchilla chọn $D/N = 20$ thay vì 1,07 và thắng rõ rệt.
- A5. Vì vài trăm tới vài nghìn ảnh chuyên ngành không đủ để học lại những thống kê ảnh cơ bản mà 140 triệu ảnh đã học hộ. Đặc trưng nền tảng cung cấp phần chung, còn phần chuyên ngành chỉ cần một đầu tuyến tính — trừ khi vật lý tạo ảnh khác hẳn, lúc đó lập luận này sụp.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn mở một xưởng và cần một người phân loại ốc vít. Cách cũ là thuê một người chưa biết gì rồi ngồi cạnh, cầm từng con ốc lên và nói tên nó, hàng nghìn lần. Rất tốn công, và bạn phải trả tiền cho từng lần nói tên.

Mẹo của mười năm qua là: đừng tuyển người chưa biết gì. Hãy nhận một người đã nhìn hàng trăm triệu bức ảnh đủ loại trong đời — xe cộ, cây cối, mặt người, nhà cửa — dù chưa từng thấy con ốc nào của bạn. Người đó đã có sẵn con mắt: biết thế nào là một cạnh, một vết xước, hai vật giống nhau. Bạn chỉ cần dạy tên gọi, và việc đó xong trong nửa buổi thay vì ba tháng.

Cái giá không nằm ở công sức của bạn mà nằm ở chỗ khác. Con mắt ấy được rèn bằng hàng trăm triệu bức ảnh và một hoá đơn tiền điện khổng lồ; chỉ là người khác đã trả hộ, và bạn không biết họ đã cho nó xem những gì. Nên khi ảnh của bạn chụp bằng máy hồng ngoại chứ không phải máy ảnh thường, kinh nghiệm kia bỗng vô dụng và không ai báo trước cho bạn.

Nửa sau của chương là một bài học về tiền: cả ngành từng tin máy càng to càng giỏi, rồi phát hiện phần lớn máy to bị bỏ dối — cùng số tiền điện, một cái máy nhỏ hơn nhưng được xem nhiều ảnh hơn thì giỏi hơn hẳn.

Chỗ không rút gọn được: vì sao con mắt rèn trên ảnh đời thường lại dùng tốt cho ảnh nội soi hay ảnh vệ tinh thì tới nay chưa ai giải thích gọn mà không sai; mọi cách nói đơn giản đều bỏ quên phần ví dụ là ảnh radar, nơi nó gần như không chuyển được.

Một câu: *Đừng dạy máy nhìn từ đầu — hãy mượn một con mắt đã được rèn bằng dữ liệu và tiền điện của người khác, rồi chỉ dạy nó tên gọi trong nghề của bạn.*

Tổng kết chương

Chương này kể một chuyển động lịch sử. Giám sát từng là thứ đắt nhất trong hệ thống; ngành đã tìm ra bốn cách lấy nó gần như miễn phí. Cái giá mỗi lần là tín hiệu gián tiếp hơn và khó kiểm chứng hơn. Kết quả tích lũy là mô hình nền tảng. Nó đổi vai trò của bạn từ người huấn luyện sang người tích hợp. Câu hỏi trung tâm đổi theo: không còn là “mô hình này có đúng không”, mà là “tôi có biết nó giả định gì và hỏng ở đâu không”.

Mặt định lượng gọn hơn nhưng tổn kém hơn nếu làm sai: compute, tham số và dữ liệu là ba mặt của một ngân sách; bài học Chinchilla là ví dụ đắt tiền nhất trong lịch sử ngành về việc tối ưu một mặt mà quên hai mặt kia; và mọi con số chung, kể cả tỉ lệ 20 *token* trên tham số, chỉ là điểm khởi đầu cho phép đo của riêng bạn.

Hai chỗ chương này để ngỏ và được nhận tiếp ở nơi khác. Thứ nhất, làm sao *đánh giá* một mô hình nền tảng cho tử tế khi bạn không biết nó đã thấy gì — đó là E/24-thi-nghiem-va-ablation và E/25-do-tin-cay-va-van-hanh. Thứ hai, một khi đã chọn được mô hình thì làm sao ép nó vào ngân sách phần cứng thật. Đó là toàn bộ Phần D, bắt đầu ở D/20-toi-uu-va-nen-mo-hinh. Ở đó cùng một logic ngân sách xuất hiện lại, chỉ khác là cột chi đổi từ compute huấn luyện sang compute **suy luận** (*inference*).

▮ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Mục 1 xếp các nguồn giám sát theo độ rẻ, mục 2 xếp chi phí theo compute và dữ liệu. Ghép hai trục lại: nguồn giám sát rẻ nhất có phải nguồn có tổng chi phí thấp nhất không, và khoản chi bị bỏ sót nằm ở đâu?
2. Augmentation của học đối lập ép một tập bất biến vào biểu diễn, còn định luật scaling nói cứ thêm dữ liệu là loss giảm. Với một tác vụ mà bất biến bị ép sai, thêm dữ liệu pretraining có cứu được không, và số hạng nào trong công thức scaling mô tả đúng chỗ không cứu được?
3. Bạn nhận một backbone mở trọng số và định dùng cho ảnh y tế. Ba câu hỏi phải trả lời trước — một về nguồn giám sát, một về nhiệm vụ dữ liệu, một về giấy phép — là gì, và câu nào bạn thật sự kiểm chứng được bằng thí nghiệm?
4. Vì sao thí nghiệm tìm điểm giao giữa linear probe và fine-tune lại dễ ra kết quả sai, và trong các cách làm sai thường gặp, cách nào không lộ ra nếu chỉ nhìn bảng kết quả cuối?
5. LoRA giả định ma trận cập nhật có hạng thấp, còn mục 1 lập luận theo lượng thông tin mới so với số tham số được phép thay đổi. Hai lập luận đó gặp nhau ở đâu, và triệu chứng nào cho biết hạng bạn chọn quá nhỏ so với lượng thông tin mới?
6. Chương nói bạn đã đổi vai từ người huấn luyện thành người tích hợp. Việc đổi vai đó làm loại lỗi nào khó phát hiện hơn hẳn so với thời tự huấn luyện, và bạn bù lại bằng phép đo nào?

Nguồn

- [1] J. Deng, W. Dong, R. Socher, et al., “ImageNet: A Large-Scale Hierarchical Image Database,” in *CVPR*, 2009.
- [2] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision,” in *ICML*, 2021.
- [3] J. Li, D. Li, S. Savarese, et al., “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *ICML*, 2023.
- [4] H. Liu, C. Li, Q. Wu, et al., “Visual Instruction Tuning,” in *NeurIPS*, 2023.
- [5] M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features Without Supervision,” *TMLR*, 2024.
- [6] S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?” In *CVPR*, 2019.
- [7] T. Chen, S. Kornblith, M. Norouzi, et al., “A Simple Framework for Contrastive Learning of Visual Representations,” in *ICML*, 2020.
- [8] K. He, H. Fan, Y. Wu, et al., “Momentum Contrast for Unsupervised Visual Representation Learning,” in *CVPR*, 2020.
- [9] J.-B. Grill, F. Strub, F. Altché, et al., “Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning,” in *NeurIPS*, 2020.
- [10] M. Caron, H. Touvron, I. Misra, et al., “Emerging Properties in Self-Supervised Vision Transformers,” in *ICCV*, 2021.
- [11] K. He, X. Chen, S. Xie, et al., “Masked Autoencoders Are Scalable Vision Learners,” in *CVPR*, 2022.
- [12] A. van den Oord, Y. Li, and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” in *arXiv:1807.03748*, 2018.
- [13] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [14] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [15] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [16] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [17] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [18] N. Houlsby, A. Giurghi, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.
- [19] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [20] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.